Matrix-Executable System Semantics

A resident-matrix proposal for batched control and analysis

STATUS
PROPOSAL

AUTHOR
RONNIE MACK

BUILD
V3 / 2026

Abstract

State-Space Programming Methodology (SSPM) asks whether a broad class of systems can be represented as transformations over a **resident state matrix** rather than repeatedly reconstructed as collections of objects, events, or solver-specific structures. Its proposed representation separates an explicit affine core from the nonlinear and order-sensitive behavior required to preserve the original update:

$$L_t = S_t A^\top + U_t B^\top + b, \qquad S_{t+1} = P_{\mathcal{M}}(L_t + H_{\text{ordered}}(S_t, U_t, L_t)) .$$

The central hypothesis is computational and operational, not merely notational. If the system state stays resident, an intervention can alter a target state, control, constraint, or operator entry and propagate its effects through shared transformations. Many alternatives can then be stacked as matrices and evaluated together. This may make model-predictive control, reachability, composition, and scenario analysis faster and more dynamically reconfigurable than rebuilding traditional system representations. An unrestricted residual makes the decomposition trivial, however. The useful claim therefore depends on sufficient affine coverage, exact semantic preservation, and a measurable advantage before residual burden dominates.

1. The proposal

Let $S_t \in \mathbb{R}^{N \times d}$ contain the named state of N entities and let $U_t \in \mathbb{R}^{N \times k}$ contain controls or exogenous inputs. SSPM treats the update as a linear-primary program with three declared layers:

1. **Affine propagation.** Matrices A and B and offset b expose the portion of the transition that can be composed, differentiated, batched, and analyzed directly.
2. **Ordered residual.** H_{ordered} retains nonlinear expressions, branches, updated-state reads, masks, and graph reductions that cannot be moved into the affine terms without changing behavior.
3. **Projection and modes.** $P_{\mathcal{M}}$ applies bounds, discrete modes, and admissibility rules explicitly rather than hiding them in backend code.

For Q candidate interventions, SSPM forms a scenario batch

$$\mathcal{S}_t = \begin{bmatrix} S_t + \Delta S_t^{(1)} \\ \vdots \\ S_t + \Delta S_t^{(Q)} \end{bmatrix},$$

then propagates shared operators across the batch. The expected gain is not that matrix multiplication makes every nonlinear system linear. It is that common structure remains resident and reusable while target changes are represented as deltas. The implementation can avoid repeated object construction, event-graph rebuilding, parsing, allocation, and host-device movement. The same representation can also expose exact composition and controllability for affine systems, branch-aware analysis for piecewise-affine systems, and local or empirical analysis for nonlinear residual systems.

This direction is adjacent to Koopman methods, which evolve observables through a linear operator and can support linear predictive control in lifted coordinates [1,2]. SSPM differs in emphasis: it begins from typed update-program semantics, retains an explicit ordered residual, and tests resident intervention economics across traditional, vectorized, fused, and accelerator implementations. Prior work also establishes an important boundary: exact finite-dimensional linear representations containing the original state are uncommon for general nonlinear dynamics [1]. SSPM therefore proposes a testable representation strategy, not universal finite-dimensional linearization.

2. What exists now

The completed V1/V2 research cycle supplies a semantic and execution substrate for this proposal. It defines typed state and input contracts, finite-value and bound checks, update-order declarations, a restricted transition IR, and generated NumPy, Numba, Torch, and C++ execution paths. Grounded bounded-queue and controlled-Kuramoto families test branching, graph coupling, projection, and nonlinear updates. The frozen V2 evidence contains 120 workload configurations and 12,080 raw timing samples; all preregistered semantic, mutation, traceability, grounded-family, and Hodgkin-Huxley gates passed.

The earlier backend-planning result is intentionally negative. A feature-aware selector reached median held-out p95 regret of 0.3159, compared with 0.2981 for a fixed Numba policy, and therefore did not satisfy its stronger generalization gate. This demotes runtime selection from the main research contribution.

These results do **not** yet validate the resident-matrix thesis. The current implementation does not extract explicit A, B, and b from source updates; it has no `AffineResidualProgram`, batched-intervention API, condensed MPC operator, or reachability interface. V1/V2 show that heterogeneous implementations can be admitted under explicit semantic contracts. They do not show that arbitrary-system updates retain enough affine structure for faster manipulation.

3. Falsifiable V3 program

V3 is separately preregistered around three workloads: an exact LTI mass-spring-damper or double integrator, a piecewise-affine bounded queue and scheduler, and a controlled Kuramoto graph whose nonlinear coupling remains in the residual. The evaluation has five linked tests.

Representability. A restricted source compiler must extract affine coefficients, lower supported nonlinear and order-sensitive operations into residual IR, and reject hidden mutation, I/O, ambiguous aliasing, unbounded loops, and unsupported side effects. Differential tests compare one-step and trajectory behavior across object, scalar-loop, vectorized, and SSPM forms.

Dynamic intervention. Initial states, control matrices, capacities, policy parameters, constraints, and coupling strengths are changed without rewriting transition code. Experiments compare rebuilding traditional objects or event structures with resident operator updates for 1, 32, 256, and 2,048 simultaneous interventions over horizons 10, 20, and 50.

Control and reachability. Exact condensed linear MPC and affine reachable sets are evaluated for the LTI family; mode-aware scenario MPC and branch-aware over-approximation are used for the queue. Kuramoto control uses local-linear or shooting methods, and its reachability results remain empirical or locally bounded. Equivalent candidate controls must produce equivalent objectives, with no additional constraint violations introduced by SSPM.

Representation ablation. Object-per-entity updates, scalar array loops, manual NumPy vectorization, equivalent discrete-event execution, fused Numba, and affine-plus-residual execution receive identical states, controls, traces, horizons, precision, and materialization boundaries. The preregistered performance gate requires at least a $2\times$ advantage over object, scalar, and equivalent discrete-event baselines at 256 or more scenarios. Comparisons against strong vectorized and fused baselines remain empirical.

Residual-burden frontier. Nonlinear density, branch entropy, graph coupling, and updated-state dependencies increase systematically. The output is a phase diagram indexed by affine coverage and residual burden. This is the decisive falsification experiment: if modest residual complexity removes the resident-state advantage, the generalized claim must narrow to systems with stronger affine structure.

4. Claim boundary

The proposal is strongest when stated precisely:

> A broad class of system updates may be usefully encoded as an affine transformation of resident matrix state plus an explicit ordered residual, enabling target interventions to propagate through shared operators and candidate scenarios to execute in batches.

"Arbitrary system" names the scope being investigated; it is not a theorem that every system has a useful finite-dimensional linear representation. A residual that absorbs the entire update would preserve expressivity while providing no analytical or computational leverage. Success therefore requires all three conditions: semantic equivalence, enough reusable affine structure, and lower intervention or analysis cost at relevant scenario counts. Negative extraction results, poor reachable-set bounds, control methods with no advantage, and residual regimes where fused custom execution wins are first-class outcomes.

The contribution sought is a matrix-executable systems methodology: one representation that makes dependencies inspectable, preserves nonlinearity without disguising it, and turns intervention from system reconstruction into resident state and operator manipulation. The V3 cycle is designed to determine where that proposition is real, where it is merely convenient notation, and where it fails.

References

[1] S. L. Brunton, B. W. Brunton, J. L. Proctor, and J. N. Kutz, "Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control," *PLOS ONE*, 11(2), e0150171, 2016. https://doi.org/10.1371/journal.pone.0150171

[2] M. Korda and I. Mezić, "Linear Predictors for Nonlinear Dynamical Systems: Koopman Operator Meets Model Predictive Control," *Automatica*, 93, 149-160, 2018. https://doi.org/10.1016/j.automatica.2018.03.046

Research status. Proposal and preregistered study design. V3 results are not yet claimed. The executable implementation remains private; the public archive contains research writing and aggregate V1/V2 evidence.